VisionGuard Dataset Training and Testing Report

Report date: 10 October 2026

Task: Object detection of six road-sign categories and three traffic-light states

Status: Training and held-out test evaluation completed

The project fine-tuned a pretrained YOLOv8n detector using a compact Roboflow dataset. The completed model achieved **91.10% mAP@50** and **72.29% mAP@50–95** on the prepared test split. These are object-detection metrics, not classification accuracy or a guarantee of performance on road videos.

Dataset Information

Source and original export

Item	Details
Dataset	Road and traffic signs recognition
Provider/workspace	ibtissammouaad, Roboflow Universe
Project identifier	road-and-traffic-signs-recognition
Version	2
Export format	YOLOv8
Original class count	20
Original exported images	2,591
Original split	2,265 training; 206 validation; 120 test
Downloaded ZIP size	79,134,648 bytes, approximately 79.13 MB
Stated dataset license	CC BY 4.0
Image size	640 × 640 pixels

The export was downloaded through the user's browser, copied from Downloads into the project and extracted. Its project ZIP was removed after extraction; the downloaded copy in Downloads was preserved. The original Mapillary, Bosch, TT100K and BDD100K downloads were not resumed for this training run.

Source: Roboflow dataset version 2. License information is recorded in the export's `data.yaml` and README. The provider's license declaration does not independently verify the rights to every underlying image.

Preparation and leakage checks

The 20 original labels were mapped to the nine requested classes. Labels for other categories were omitted, while images without a selected object were retained with empty label files as background examples. The preparation code handles polygon annotations by deriving enclosing bounding boxes where a selected polygon is present. The original export is preserved separately from the prepared copy.

To reduce split leakage, preparation compared image SHA-256 hashes and original filename identifiers before Roboflow's `.rf.` suffix. Test images were reserved first, followed by validation images, then training images. **Ten**

training images and one validation image were excluded because they overlapped with a higher-priority split. A separate validation pass confirmed that no identical image bytes remained across splits. This does not establish that all visually similar frames or scenes are independent.

Prepared split	Images	Images with selected objects	Background images	Selected object instances
Training	2,255	918	1,337	1,011
Validation	205	81	124	92
Test	120	52	68	59
Total	**2,580**	**1,051**	**1,529**	**1,162**

An image can contain more than one object. Therefore, object counts differ from image counts.

Selected classes and annotations

YOLO detection labels contain five values per object: `class_id x_center y_center width height`. Coordinates are normalized to the image dimensions. Class IDs were remapped to the order below; speed limits are interpreted as km/h.

ID	Class	Training objects	Validation objects	Test objects
0	Stop sign	105	13	4
1	Yield sign	162	12	7
2	No entry	186	12	8
3	Speed limit 40	81	9	7
4	Speed limit 60	78	6	7
5	Speed limit 80	90	4	4
6	Red traffic light	123	17	13
7	Yellow traffic light	78	4	4
8	Green traffic light	108	15	5

Source preprocessing and augmentation

The source version metadata specifies resizing by stretching to 640 × 640 pixels, three training image versions, saturation adjustment up to 69%, brightness adjustment up to 59%, exposure adjustment up to 34% and blur up to one pixel. These operations were already part of the exported version. Disabling additional colour augmentation in this training run does not undo those existing transformations.

Training Algorithm and Configuration

Transfer learning with YOLOv8n

YOLOv8n is a single-stage object detector. Its backbone extracts image features, its feature-fusion layers combine information at different scales, and its detection head predicts bounding boxes and object classes. This

run used the project's existing COCO-pretrained `yolov8n.pt` as the starting checkpoint and adapted its detection head to nine classes.

The training log records **319 of 355 weight items transferred** from the pretrained checkpoint. The model contained approximately **3.01 million parameters**. The distribution-focal-loss projection layer was frozen by Ultralytics; the remaining trainable layers were fine-tuned. Training optimized bounding-box regression, class prediction and distribution focal loss. Validation was performed during training, and the best checkpoint was selected using validation fitness rather than held-out test results.

Setting	Actual value used
Model/task	YOLOv8n object detection
Epochs requested/completed	50 / 50
Input size	640 × 640
Batch size	8
Optimizer	AdamW, automatically selected by Ultralytics
Initial optimizer learning rate	0.000769
AdamW momentum/beta1 setting	0.9
Weight decay	0.0005 for decayed weight groups
Warm-up	3 epochs
Early-stopping patience	12 epochs; training reached all 50 epochs
Seed	42
Deterministic mode	Enabled
Mixed precision	Enabled; AMP checks passed
Data-loader workers	2
Image cache	Disabled
Training hardware	NVIDIA GeForce RTX 3060 Laptop GPU, 6 GB VRAM
Python	3.10.11
PyTorch/CUDA build	2.5.1+cu124
Ultralytics	8.3.105

Although `args.yaml` lists the default `lr0=0.01` and `momentum=0.937`, the log explicitly states that `optimizer=auto` ignored those defaults and selected the AdamW settings above. The final logged learning rate was approximately 0.0000229162.

Additional hue and saturation augmentation, horizontal flips and vertical flips were disabled to reduce changes to signal colours and sign orientation. Brightness augmentation remained enabled (`hsv_v=0.4`), together with translation (`0.1`), scaling (`0.5`) and mosaic (`1.0`). Mosaic was disabled for the last 10 epochs. Mixup was disabled.

Training Results

All 50 epochs completed. The epoch history records **2,108.79 seconds**, approximately **35.15 minutes**, of training. The trainer reports this as 0.586 hours. This time excludes dataset download, preparation and the separate held-out test evaluation.

Loss and validation progress

The validation metrics below are from the epoch history. Validation is used for model selection and is separate from the final test evaluation.

Metric	Epoch 1	Epoch 50
Training box loss	1.08439	0.37886
Training classification loss	4.84579	0.39857
Training distribution focal loss	1.39427	0.91262
Validation box loss	1.02259	0.63889
Validation classification loss	5.16011	0.50025
Validation distribution focal loss	1.46186	0.96672
Validation mean precision	56.61%	91.41%
Validation mean recall	40.45%	90.23%
Validation mAP@50	40.70%	92.11%
Validation mAP@50–95	27.12%	74.20%

Loss values are optimization quantities, not percentages or accuracy scores. Lower losses and improving validation metrics show progress on this dataset; they do not establish generalization to unseen road conditions.

The highest validation fitness occurred at **epoch 50**. This uses the Ultralytics validation fitness weighting of 0.1 × mAP@50 + 0.9 × mAP@50–95. The highest individual validation mAP@50 in the epoch history was 92.16%; selecting the best checkpoint considers both mAP measures.

The saved `best.pt` checkpoint occupies **6,251,811 bytes**, approximately **6.25 MB**, after the optimizer state was stripped.

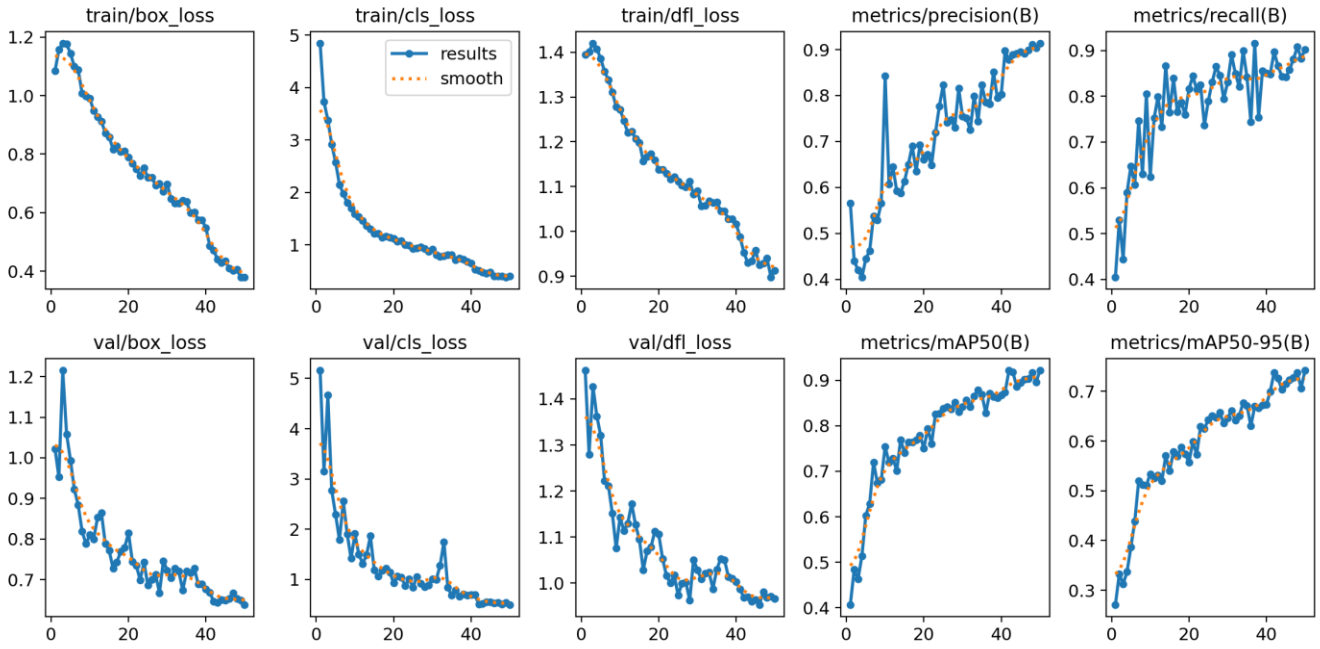


Figure 1. The trainer's saved loss and validation curves for the completed run.

Testing Results

Held-out test protocol

After training, the saved `best.pt` checkpoint was evaluated on the prepared test split at 640-pixel input resolution using the same GPU. This split contained **120 images and 59 selected object instances**, including 68 background images. The test split was not passed to the training loop or used to select the checkpoint.

The saved evaluator reports mean precision and recall across the selected classes. Average precision summarizes precision–recall performance across confidence thresholds. **mAP@50** evaluates detections at intersection-over-union (IoU) 0.50, while **mAP@50–95** averages results over IoU thresholds from 0.50 to 0.95. These results are not an evaluation at the web app's fixed confidence threshold of 0.4.

Overall test metric	Result
Mean precision	**82.49%**
Mean recall	**86.31%**
mAP@50	**91.10%**
mAP@50–95	**72.29%**

Per-class test results

Per-class values are reproduced from the trainer's rounded console output. Object counts are the test ground-truth counts.

Class	Test objects	Precision	Recall	AP@50	AP@50–95
Stop sign	4	90.9%	100.0%	99.5%	89.1%
Yield sign	7	93.9%	100.0%	99.5%	84.0%
No entry	8	94.3%	100.0%	99.5%	86.5%
Speed limit 40	7	100.0%	85.4%	87.6%	67.6%
Speed limit 60	7	70.2%	57.1%	85.3%	79.6%
Speed limit 80	4	37.9%	62.1%	60.2%	54.7%
Red traffic light	13	100.0%	72.2%	94.2%	46.2%
Yellow traffic light	4	77.7%	100.0%	94.5%	61.0%
Green traffic light	5	77.6%	100.0%	99.5%	81.9%

Speed limit 80 had the lowest AP@50 and precision in this test. Speed limit 60 also had relatively low recall. Red traffic lights showed a larger drop under stricter IoU evaluation, with AP@50–95 of 46.2%. These observations identify areas for additional examples and error inspection; the small class counts prevent strong conclusions about real-world reliability.

The evaluator measured approximately **1.4 ms preprocessing, 6.1 ms inference and 2.0 ms postprocessing per image** on this GPU. These are evaluator timings, not end-to-end video playback latency or Roboflow cloud API latency.

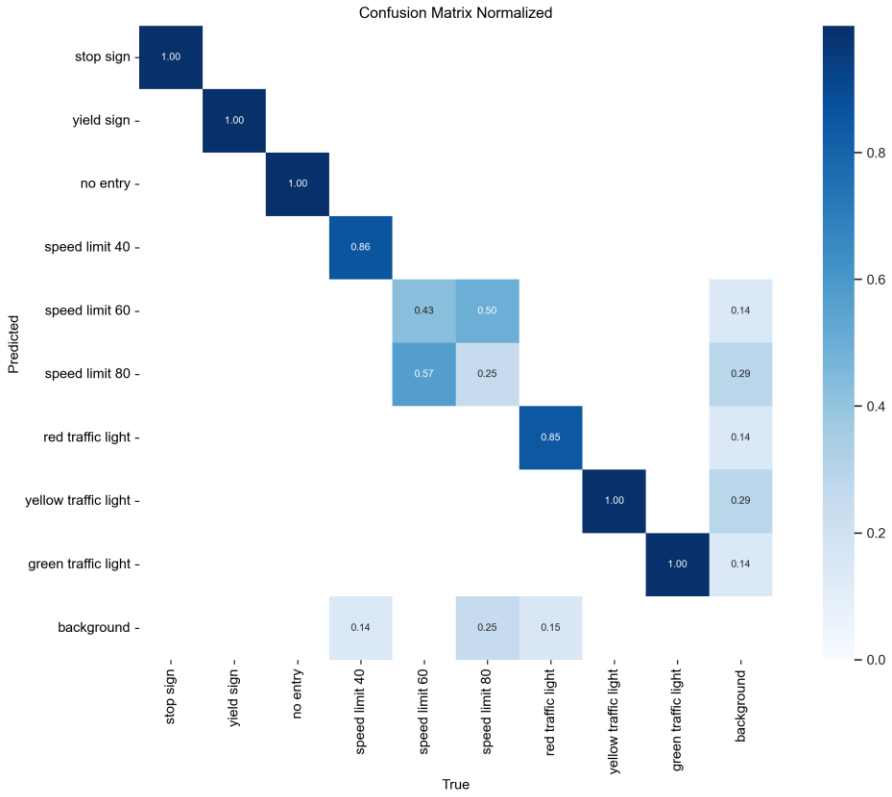


Figure 2. The saved normalized test confusion matrix, including the background category.

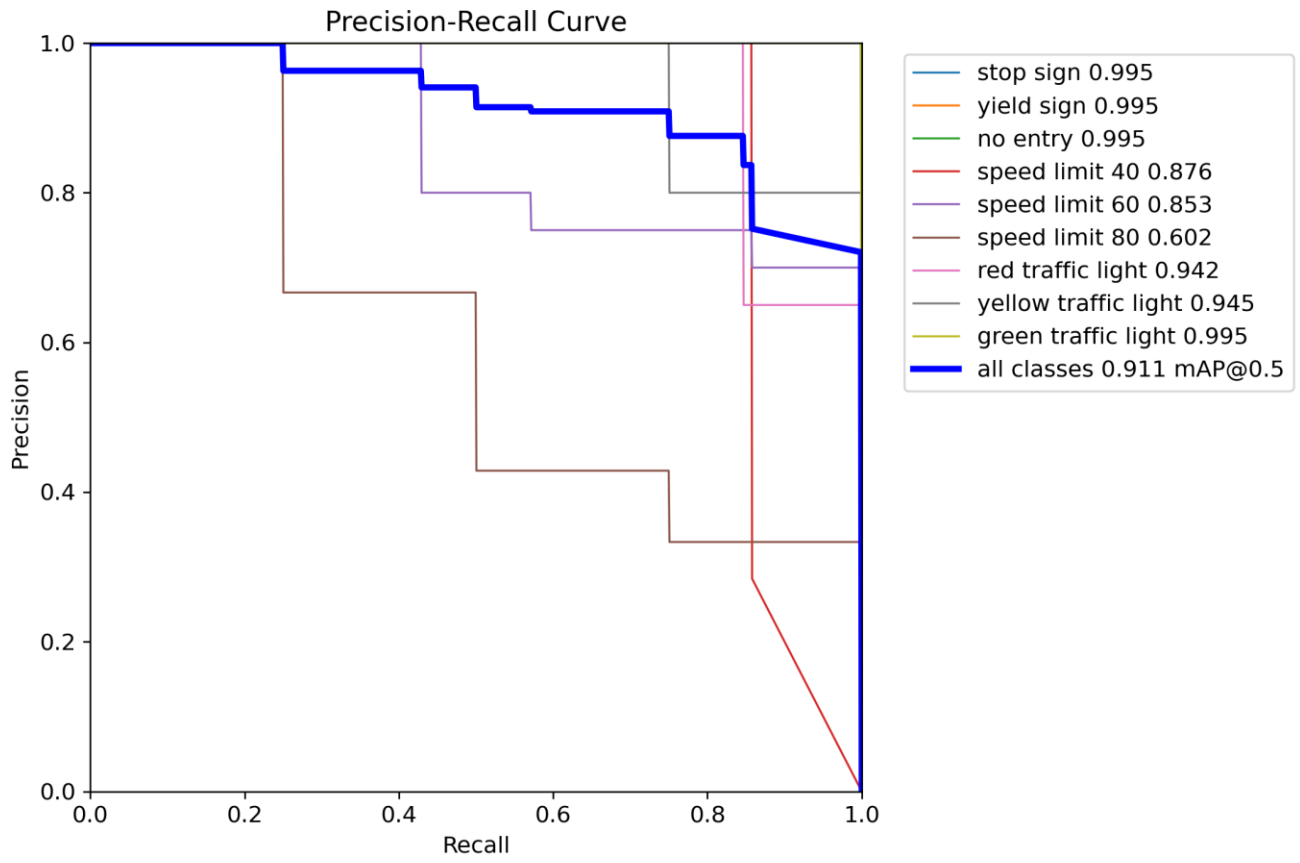


Figure 3. Precision–recall curves from the held-out test evaluation.

Separate hosted-model demonstration

The existing Roboflow-hosted model `road-and-traffic-signs-recognition/2` was also run on one public dataset image. It returned one stop-sign detection with confidence **0.9699**, approximately 97%. This was a demonstration of the cloud API integration, **not a test result of the newly fine-tuned local checkpoint**. A prediction confidence is not a measure of model accuracy. The API key was supplied through a process environment variable, and header-only authentication was verified.

Limitations and Next Steps

1. **Small test support.** Some classes have only four test objects. Per-class percentages may change substantially with a few additional cases.
2. **Source similarity.** Exact image overlap and matching source identifiers were filtered, but near-duplicates, shared locations and related recording sequences were not comprehensively audited.
3. **Light-colour augmentation.** The source export already contains saturation, brightness and exposure transformations. Traffic-light labels and colours should be visually reviewed before expanding training.
4. **Restricted scope.** The checkpoint supports only the nine selected categories. It does not cover all signs, traffic-light arrows, off states, flashing states or which signal applies to the vehicle's route.
5. **Road-video testing remains outstanding.** No independent driving-video benchmark, night/rain evaluation, small/distant-object study or end-to-end application latency measurement was completed in this run.
6. **Deployment remains separate.** The final checkpoint is saved in the training output directory. Local web-testing Python files were not edited as part of this report or the completed training/inference work.

The next validation step is to test independent road footage, particularly examples of speed limits 60/80 and small red lights. Additional examples should be collected from different journeys and conditions, with splits assigned by recording sequence. Tune operating confidence using validation data and preserve an untouched independent test set for final assessment.

Sources and Reproducibility

This report was compiled from recorded project artifacts rather than estimated results. No new training or test run was required to produce it.

Evidence	Local artifact
Dataset provenance, object counts and excluded overlaps	source.json
Prepared class names and split paths	Dataset YAML
Actual training configuration	args.yaml
Epoch history and validation metrics	results.csv
Optimizer selection, duration and per-class evaluation	Training log
Precise overall test metrics	test_metrics.json
Final trained checkpoint	best.pt
Dataset preparation implementation	prepare_roboflow.py
Training and evaluation implementation	train_limited.py
Separate hosted inference demonstration	Prediction JSON

The recorded training command used the following settings. Use a new output directory for a future run; do not overwrite this run's evidence.

```
py -3.10 training/train_limited.py `
  --data datasets/roboflow-v2/selected-nine-clean/data.yaml `
  --output training_runs/local_roboflow_v2 `
  --epochs 50 `
  --imgsz 640
```

The linked figures were copied from the saved Ultralytics outputs into this report's `assets` directory. Private API keys and signed download tokens are not included in the report.